

Tema 5. Paralelismo de datos

Arquitectura de computadores - Semana 11

Grado en *Ingeniería Informática*

Departamento de Informática. Universidad de Jaén



Objetivos

General

Conocer las arquitecturas de procesador para el tratamiento de datos en paralelo, sus diferencias, ventajas e inconvenientes

Específicos

- Saber **en qué consiste** el paralelismo de datos
- Conocer los **orígenes** del paralelismo de datos
- Identificar las **soluciones existentes** para procesar datos en paralelo
- Diferenciar entre las **categorías de paralelismo de datos** actuales
- Analizar las **ventajas e inconvenientes** entre las arquitecturas paralelas

Tipos de paralelismo



Taxonomía de Flynn

Objetivo

Clasificación de las arquitecturas de procesador propuesta por Michael J. Flynn, según el número de flujos de instrucciones y de datos

Taxonomía

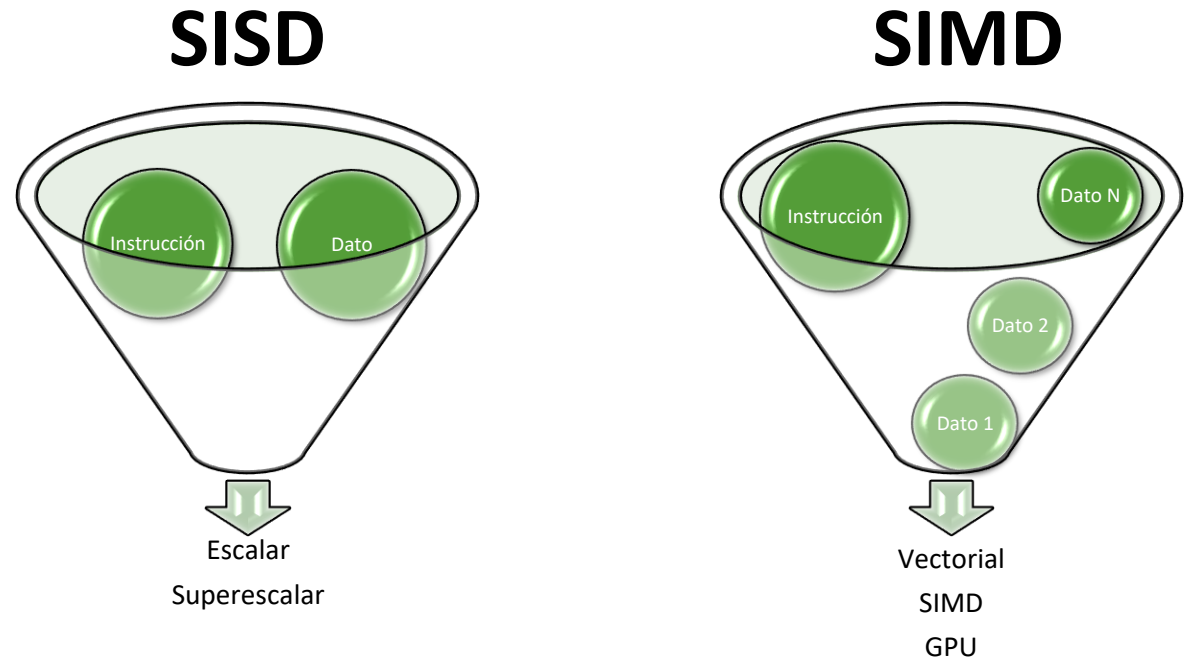
		Instrucciones	
		Una	Varias
Datos	Uno	SISD	MISD
	Varios	SIMD	MIMD

SISD: *Single Instruction Single Data*

MISD: *Multiple Instructions Single Data*

SIMD: *Single Instruction Multiple Data*

MIMD: *Multiple Instructions Multiple Data*



La mayoría de cálculos científicos, físicos y matemáticos operan sobre **matrices y vectores de datos** de idéntico tamaño, por lo que las arquitecturas SIMD resultan de interés para dichas tareas

Procesadores SISD

Introducción

Son los procesadores que se ajustan a la arquitectura Von Neumann y que procesan instrucciones asociadas a un dato de tipo escalar

Implementaciones

- Los procesadores **escalares**, con cauce segmentado o no, entran en esta categoría, al captar una sola instrucción que actúa sobre un dato: carga, almacenamiento, aritmética, etc.
- También los procesadores **superescalares** son SISD
 - Se captan múltiples instrucciones en un ciclo de reloj
 - El cauce superescalar es capaz de ejecutar varias instrucciones en paralelo
 - Cada instrucción tiene **operandos escalares** de entrada y de salida

En general, procesar múltiples datos precisa escribir un bucle que los recorra

Procesadores SIMD

Introducción

Son procesadores con capacidad para ejecutar una misma operación sobre múltiples datos de manera simultánea, sin necesidad de bucles

Implementaciones

Hay distintos tipos de implementación para el procesamiento paralelo de datos:

- **Procesadores vectoriales:** cuentan con instrucciones y registros específicos para el trabajo con vectores de datos de longitud arbitraria
- **Procesadores con extensiones SIMD:** disponen de instrucciones y unidades funcionales que facilitan operaciones sobre vectores de longitud fija (se conoce como *packed SIMD* para diferenciarlo de la denominación general SIMD)
- **Procesadores con paralelismo masivo de datos (GPU):** miles de unidades funcionales sencillas, capaz de ejecutar cada una de ellas un hilo sobre un dato de longitud fija (modelo SIMT: *Single Instruction Multiple Thread*)

Paralelismo de datos

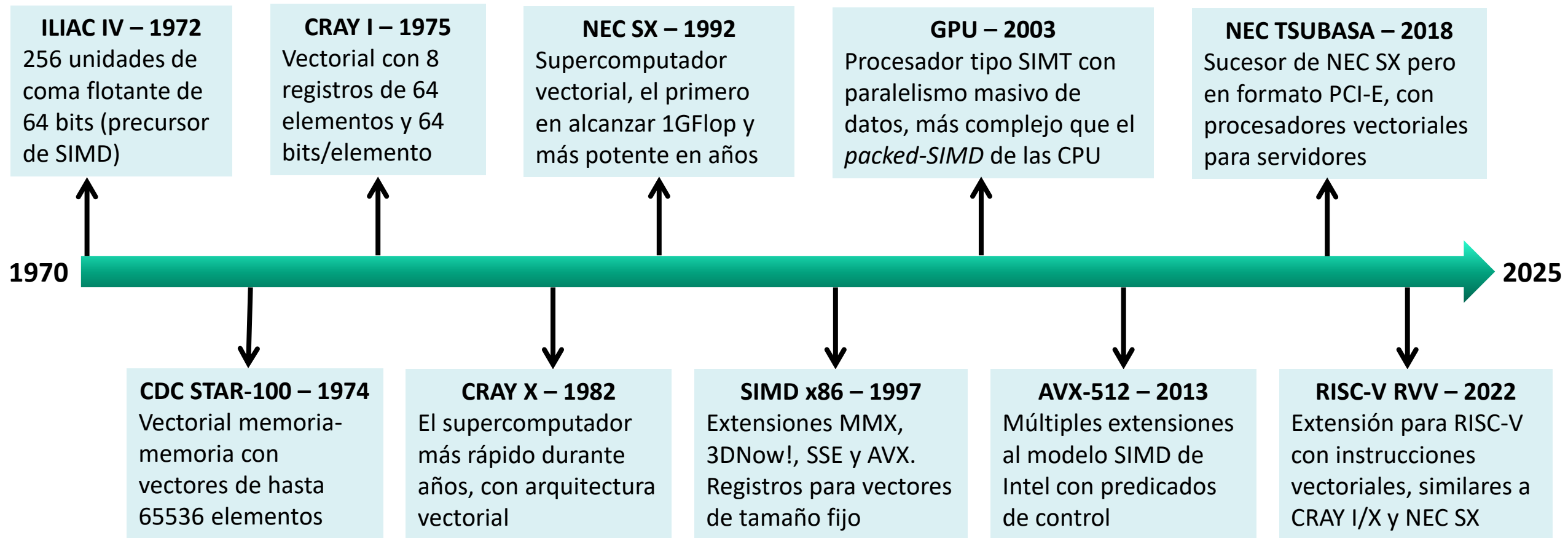
Fundamentación

Una gran parte de programas de ordenador operan sobre colecciones de datos dispuestos secuencialmente en memoria (vectores), por lo que una mejora para ese tipo de operaciones redundaría en un mejor rendimiento

Conceptos

- **Paralelismo:** la mayoría de operaciones entre vectores no tienen dependencias de datos entre elementos, por lo que pueden ejecutarse en paralelo, ya sea con múltiples unidades funcionales (SIMD) o en un cauce segmentado (vectorial)
- **Cuello de botella de Flynn:** al operar sobre múltiples datos con una sola instrucción, sin requerir bucles, se evitan los riesgos de control origen del conocido como cuello de botella de Flynn
- **Anchura de banda:** se reduce en el bus de instrucciones pero se incrementa en el bus de datos. Las instrucciones vectoriales presentan patrones de acceso muy predecibles, en posiciones secuenciales, por lo que resulta fácil optimizarlas. La latencia inicial se ve compensada por la transferencia del vector completo, en contraposición a accesos a datos individuales
- **SIMD:** replica las unidades de cálculo (ALU), sin necesidad de replicar las unidades de control, de forma que una sola instrucción se aplica sobre múltiples datos en paralelo
- **Vectorial:** cauce segmentado profundo que hace posible operar sobre múltiples datos en paralelo con una sola unidad de cálculo (o un número limitado de ellas) siempre y cuando no haya dependencias entre datos

Cronología del paralelismo de datos



1976

CRAY 1

160 MFlops



Procesadores vectoriales



Procesadores vectoriales

Introducción

*Procesadores con registros y unidades funcionales específicas para operar sobre vectores de elementos de forma paralela, denominados durante años **supercomputadores***

Funcionamiento

- **Precursores:** el [CDC STAR-100](#), [TI-ASC](#) y [Cray-1](#)
- **Memoria:** el CDC operaba directamente sobre **vectores en memoria**, diseño no escalable por el cuello de botella que representaba el acceso a la misma
- **Registros:** la incorporación de **registros vectoriales** en el Cray-1 representó una mejora fundamental al evitar múltiples accesos a memoria para operar sobre los mismos datos
- **Arquitectura:** estos procesadores cuentan con un conjunto de registros de configuración de los vectores sobre los que operan, así como con unidades funcionales segmentadas profundas (*deep pipelining*)

Almacenamiento de los datos

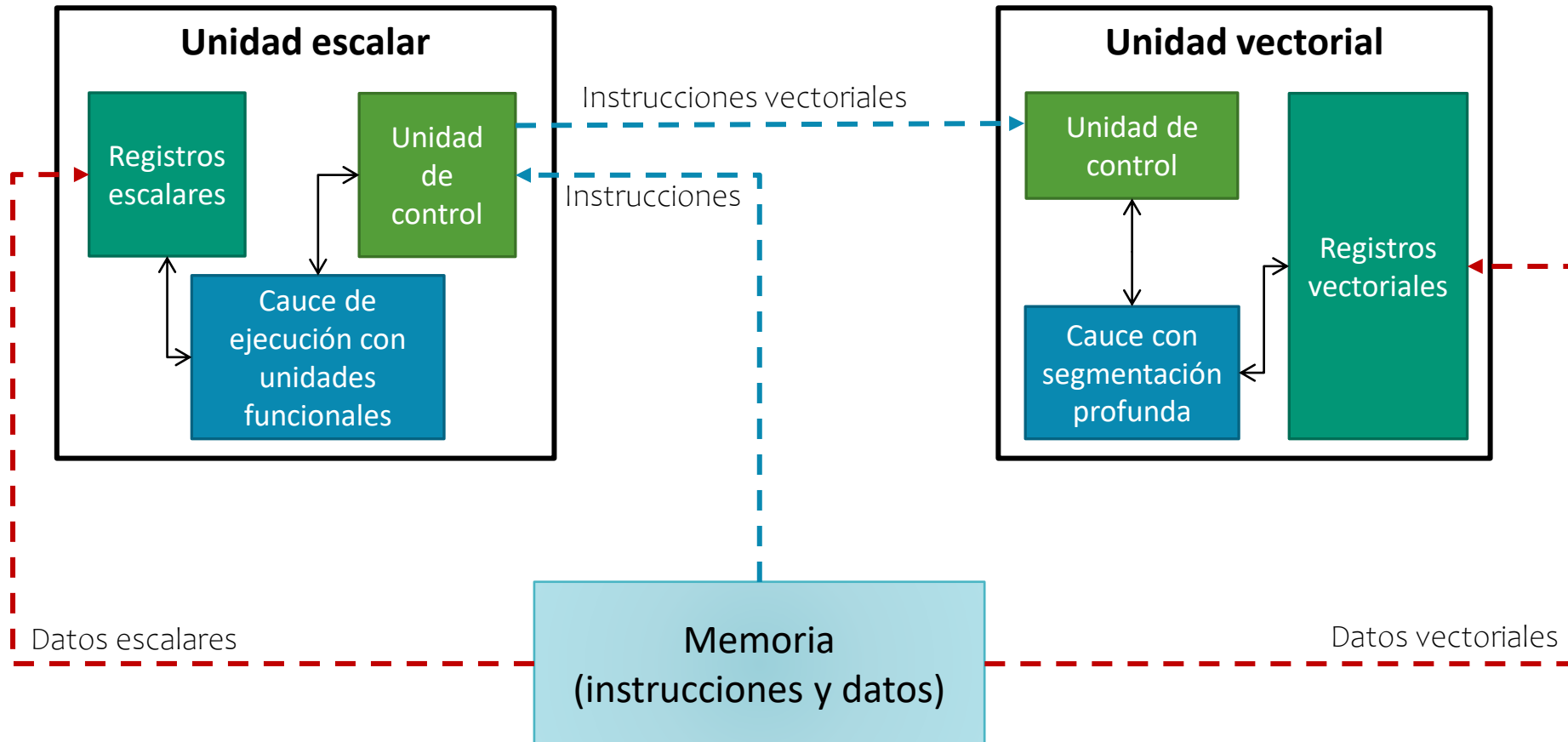
Razonamiento

Los procesadores vectoriales operan en conjuntos de datos (vectores) con un número variable de elementos y de tamaño por elemento (tipo)

Categorías

- **Memoria-Memoria:** la usada por ordenadores como el CDC STAR-100 y el TI-ASC
 - Los operandos origen y destino de una operación vectorial residen en memoria principal, desde donde van cargándose y almacenándose a medida que se ejecuta una operación
 - No hay, a priori, más limitación del tamaño de los vectores que la impuesta por la memoria disponible
 - Su rendimiento es relativamente bajo por los accesos continuos a memoria
- **Registros vectoriales:** la usada por Cray-1, Fujitsu VP-200 y los vectoriales modernos
 - Cuentan con un banco de registros vectoriales, análogo al banco de registros para datos enteros y en punto flotante, en los que se cargan los datos sobre los que se operará
 - Los registros de datos vectoriales tienen un cierto tamaño establecido por el diseño del procesador
 - Su rendimiento es mucho más elevado que el de los procesadores vectoriales memoria-memoria
 - El diseño hardware es más complejo y caro respecto a los vectoriales memoria-memoria

Arquitectura de bloques de un procesador vectorial

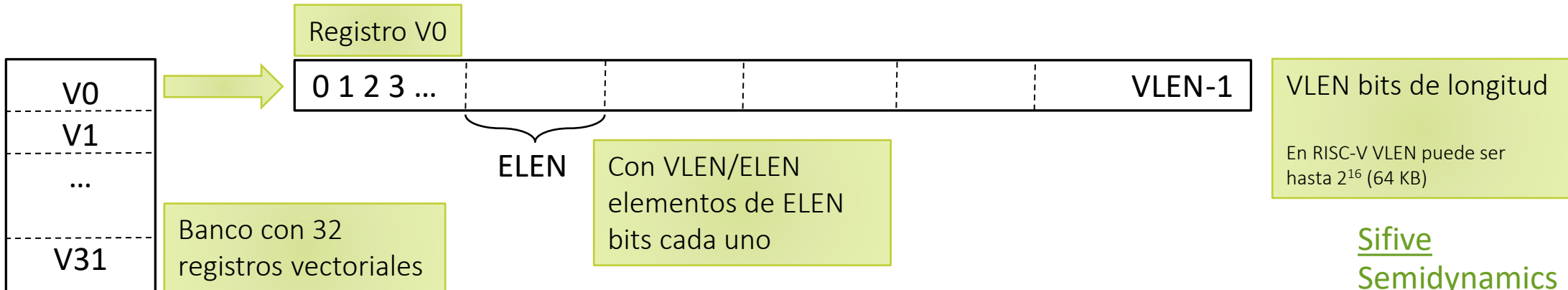


- **Unidad escalar:** es la que ya conocemos, con cauce segmentado simple o superescalar. Se caracteriza por un conjunto de registros cada uno de los cuales almacena un solo valor, ya sea un número entero o en punto flotante
- **Unidad vectorial:** es la que nos interesa estudiar en este apartado para conocer la estructura del banco de registros vectoriales y el funcionamiento tanto de la unidad de control como del cauce con segmentación profunda

Registros vectoriales – RISC-V

Características

- **Número:** contamos con un banco de registros vectoriales, similar al de los registros escalares, con un número de registros variable según el procesador concreto
 - El Cray-1 contaba con 64 registros vectoriales
 - Los modernos procesadores tienen un número variable, habitualmente es 32 o más
- **Tamaño:** diferenciar entre el tamaño total del registro y el tamaño de elemento
 - El tamaño en bits de un registro vectorial viene dado por la implementación concreta de un procesador. A este factor se le suele llamar **VLEN** y lo habitual es que sea de 64, 128, 256 o 512 bits
 - En un registro vectorial se almacenan múltiples datos de tamaño **ELEN**, parámetro que suele ser configurable entre 8, 16, 32 y 64 bits
 - El cociente **VLEN/ELEN** indica el número de elementos que es posible almacenar en un registro vectorial



Tamaño de los registros vectoriales – RISC-V

Agrupamiento

- **LMUL:** este parámetro determina el número de registros que se agrupan para formar uno de mayor tamaño
 - Puede tomar los valores 1, 2, 4 y 8
- **Registros disponibles:** al agrupar, se cuenta con un menor número de registros
 - Los registros alcanzan hasta 8 veces el **VLEN** original, lo cual permite operar sobre vectores mucho mayores, algo que no es posible con otras arquitecturas como las extensiones SIMD
 - Se reduce el número de iteraciones para realizar el mismo trabajo, por lo que hay un menor número de riesgos de control en el programa

LMUL	Registros vectoriales disponibles																																		
m1	v0	v1	v2	v3	v4	v5	v6	v7	v8	v9	v10	v11	v12	v13	v14	v15	v16	v17	v18	v19	v20	v21	v22	v23	v24	v25	v26	v27	v28	v29	v30	v31			
m2	v0		v2		v4		v6		v8		v10		v12		v14		v16		v18		v20		v22		v24		v26		v28		v30				
m4	v0				v4				v8				v12				v16				v20				v24				v28						
m8	v0								v8								v16								v24										

Unidad de control vectorial

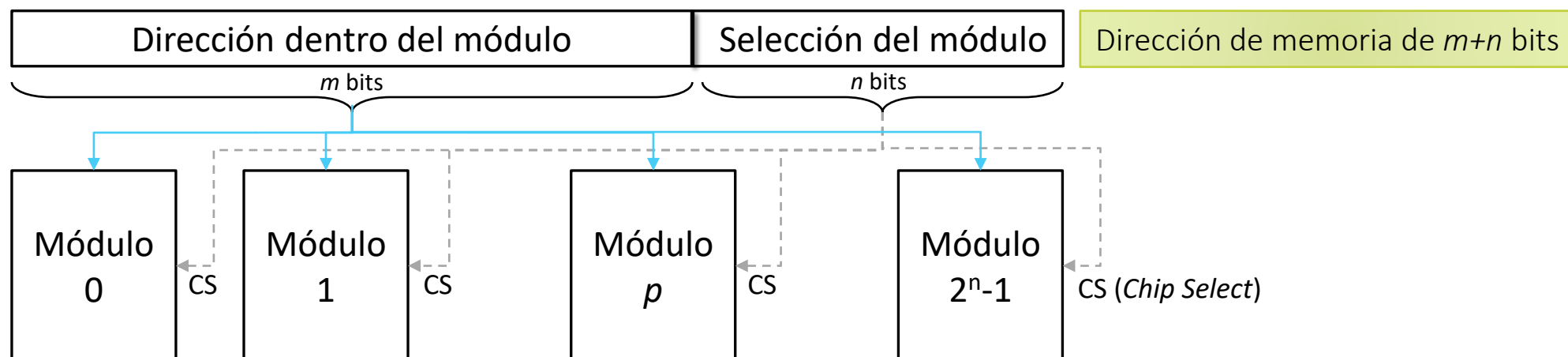
Funcionamiento

- **Paralelismo:** la unidad de control (UC) tiene un diseño específico
 - La UC escalar gestiona el **paralelismo a nivel de instrucción** con múltiples instrucciones en un cauce
 - La UC vectorial gestiona el **paralelismo de datos** con una sola instrucción operando sobre un conjunto de datos
- **Registros de control:** además de los registros de datos, la UC cuenta con registros internos para controlar la operación de las unidades vectoriales
 - Un registro establece el **tamaño de cada elemento** de datos almacenado en los registros vectoriales y otro la **longitud de los vectores** con lo que se trabajará (puede ser mayor o menor que el tamaño de los registros vectoriales)
 - Un registro determina si se formarán **grupos de registros vectoriales** para operar con vectores de mayor tamaño que los considerados por el hardware
 - Los **registros de enmascaramiento** determinan a qué elementos de los registros vectoriales afectará la operación y qué ocurrirá con aquellos no afectados
- **Detección de riesgos:** la UC ha de verificar que la operación es posible
 - Los datos han de estar **correctamente alineados** en memoria para poder operar sobre ellos
 - No han de existir **dependencias entre elementos** del vector para poder ejecutar la operación en paralelo

Cargas y almacenamientos

Conceptos

- **Disposición lógica:** las instrucciones de carga y almacenamiento de registros vectoriales son análogas a las escalares, con aspectos específicos
- **Memoria entrelazada:** se usa para acelerar la transferencia de múltiples elementos
 - El entrelazamiento de orden bajo usa los **bits de menor peso de la dirección** para seleccionar el módulo de memoria desde el que se hace la transferencia, lo cual permite su paralelización
 - Se opera con direcciones de memoria consecutivas, pero también contempla la **carga con stride** para elementos no dispuestos en direcciones consecutivas pero con un espaciado uniforme
 - El almacenamiento de resultados **puede enmascarse**, de forma que solo ciertos elementos del registro vectorial sean efectivamente escritos en memoria

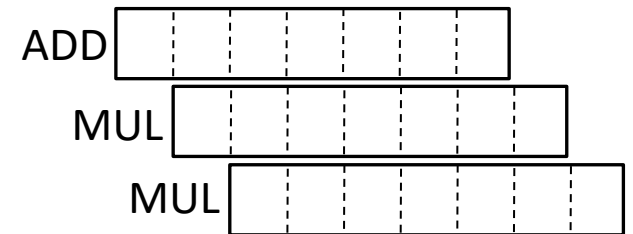
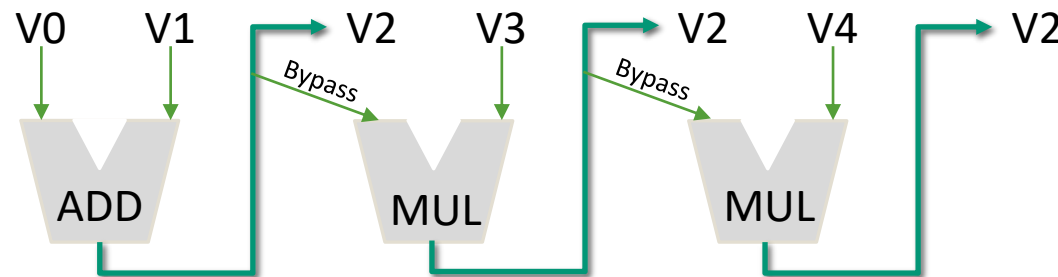


Unidades funcionales vectoriales

Arquitectura

- **Unidades específicas:** hay una unidad para cada tipo de operación, en lugar de una sola ALU de propósito general. Pueden replicarse formando *lanes* para mayor capacidad
- **Segmentación:** las unidades funcionales están muy segmentadas, lo cual les permite operar en paralelo sobre múltiples elementos de datos de un vector
 - Sumas, restas y multiplicaciones en 5-8 etapas
 - Divisiones con segmentación de hasta 20 etapas
- **Encadenamiento:** efectuar múltiples operaciones consecutivas sobre los registros
 - Una vez cargados los datos en los registros vectoriales, es posible efectuar **múltiples operaciones** sobre ellos sin esperar a que se complete una operación previa
 - El encadenamiento de operaciones **reduce el número de esperas** y, por tanto, mejora el rendimiento

```
vadd.vv v2, v0, v1
vmul.vv v2, v2, v3
vmul.vv v2, v2, v4
```



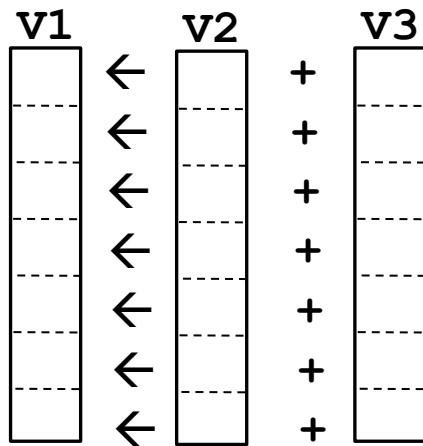
Operaciones vectoriales I – RISC-V

Tipos de operandos: el segundo operando puede ser un vector, un registro escalar o un valor inmediato y se genera como resultado un vector de valores.

Tipos de operaciones: adición, sustracción, producto, división, mínimo, máximo, tanto de enteros como punto flotante. Desplazamiento de bits y operaciones lógicas a nivel de bit para enteros.

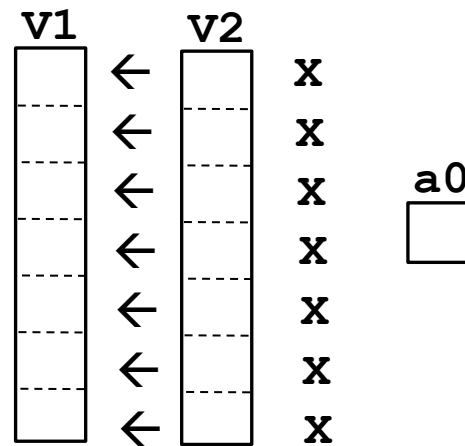
Entre pares de elementos

vadd.vv v1, v2, v3 $\rightarrow v1[i] = v2[i] + v3[i]$



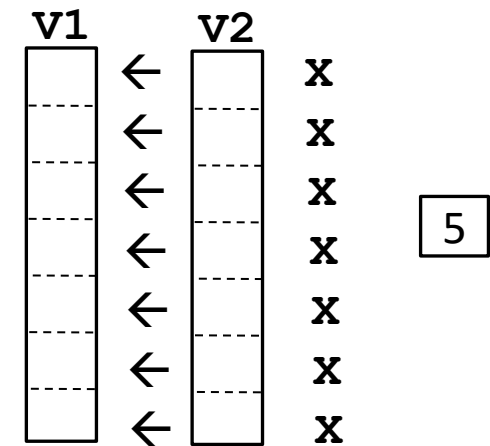
Entre vector y escalar

vmul.vx v1, v2, a0 $\rightarrow v1[i] = v2[i] \times a0$



Entre vector e inmediato

vmul.vi v1, v2, 5 $\rightarrow v1[i] = v2[i] \times 4$



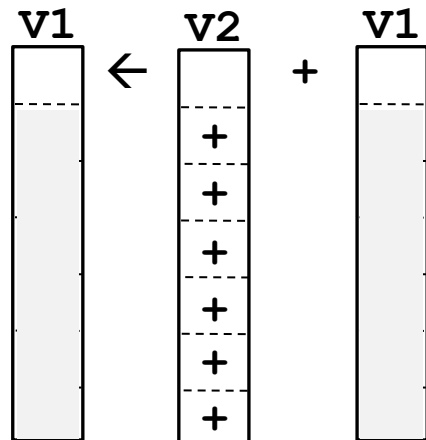
Operaciones vectoriales II – RISC-V

Tipos de operandos: los tres operandos son registros vectoriales. En el primer elemento del destino se acumula el resultado de la operación aplicada a todos los elementos del segundo más el primer elemento del tercero

Tipos de operaciones: suma, máximo, mínimo, operaciones lógicas entre bits.

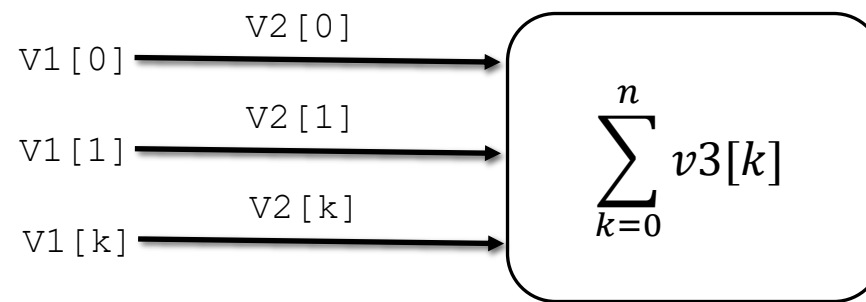
Reducción

vredsum.vs v1, v2, v1



Ejemplo perceptrón

vle32.v v1, t0 # Entradas
vle32.v v2, t1 # Pesos
vmul.vv v3, v1, v2 # Entradas X Pesos
vredsum.vs v1, v3, v0 # Suma total



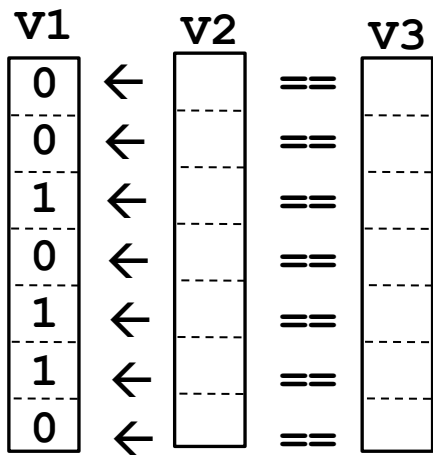
Operaciones vectoriales III – RISC-V

Enmascaramiento: una operación no actúa sobre todos los elementos del vector, sino solo sobre aquellos indicados por una máscara

Creación de la máscara: mediante operaciones relacionales entre elementos y operaciones lógicas entre los bits de máscaras

Comparar vectores

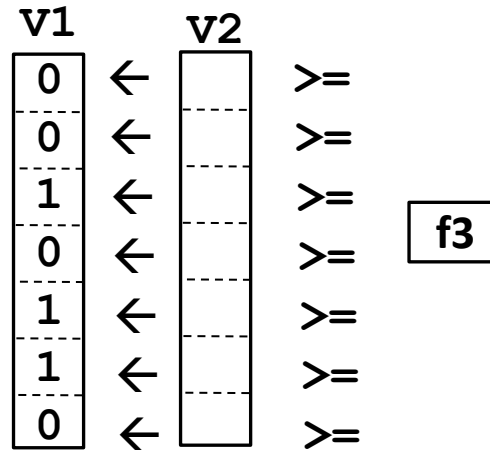
vmseq.vv v1, v2, v3



Poner a 1 en la máscara los elementos que cumplen $v2[i] == v3[i]$

Comparar vector y escalar

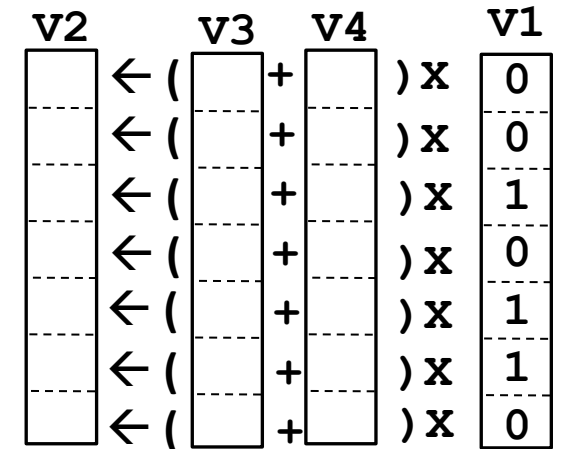
vmfge.vf v1, v2, f3



Poner a 1 en la máscara los elementos que cumplen $v2[i] \geq f3$

Operación enmascarada

vadd.vv v2, v3, v4, v1



Sumar aquellos pares de elementos para los que la máscara está activa

Extensiones SIMD



Extensiones SIMD

Introducción

Procesadores con registros de mayor tamaño que permiten almacenar paquetes de datos y operar sobre ellos de manera conjunta

Principio de funcionamiento

- **Registros:** cuentan con un cierto número de registros extendidos de mayor tamaño, entre 64 bits y 512 bits según la implementación
- **Arquitectura:** el paralelismo viene dado por la operación de las unidades funcionales sobre todos los datos almacenados en los registros (*packed SIMD*), en lugar de por la segmentación profunda de dichas unidades
- **Tamaño de vector:** el tamaño de los vectores sobre los que pueden operar es fijo, al no existir registros de control para establecer su longitud
- **Complejidad:** el tamaño de los datos empaquetados en los registros viene codificado en las instrucciones, por lo que se agrega un gran número de ellas

El origen de las extensiones SIMD

Nacimiento

Estas extensiones surgen en 1996 con la denominación Intel MMX para acelerar operaciones sobre operandos de tamaño reducido

Principio de funcionamiento

- **Registros FPU:** los procesadores de la época contaban con una FPU que disponía de varios registros de 64 bits para efectuar operaciones en coma flotante
- **Operaciones multimedia:** muchas operaciones con datos gráficos y de audio se llevaban a cabo con enteros de 8 o 16 bits, para lo que se recurría a los GPR
- **Extensiones MMX:** Intel introduce un conjunto de instrucciones que, apoyándose en los registros FPU de 64 bits, permiten operar de forma simultánea sobre ocho datos de 8 bits o cuatro datos de 16 bits
- **Limitaciones:** no es posible realizar cálculos de coma flotante mientras se ejecutan instrucciones MMX, ya que usan el mismo conjunto de registros

Implementaciones SIMD

Arquitecturas

Las extensiones SIMD surgieron para la arquitectura x86 y es donde han evolucionado, aunque en la actualidad hay implementaciones para otras arquitecturas como ARM o RISC-V

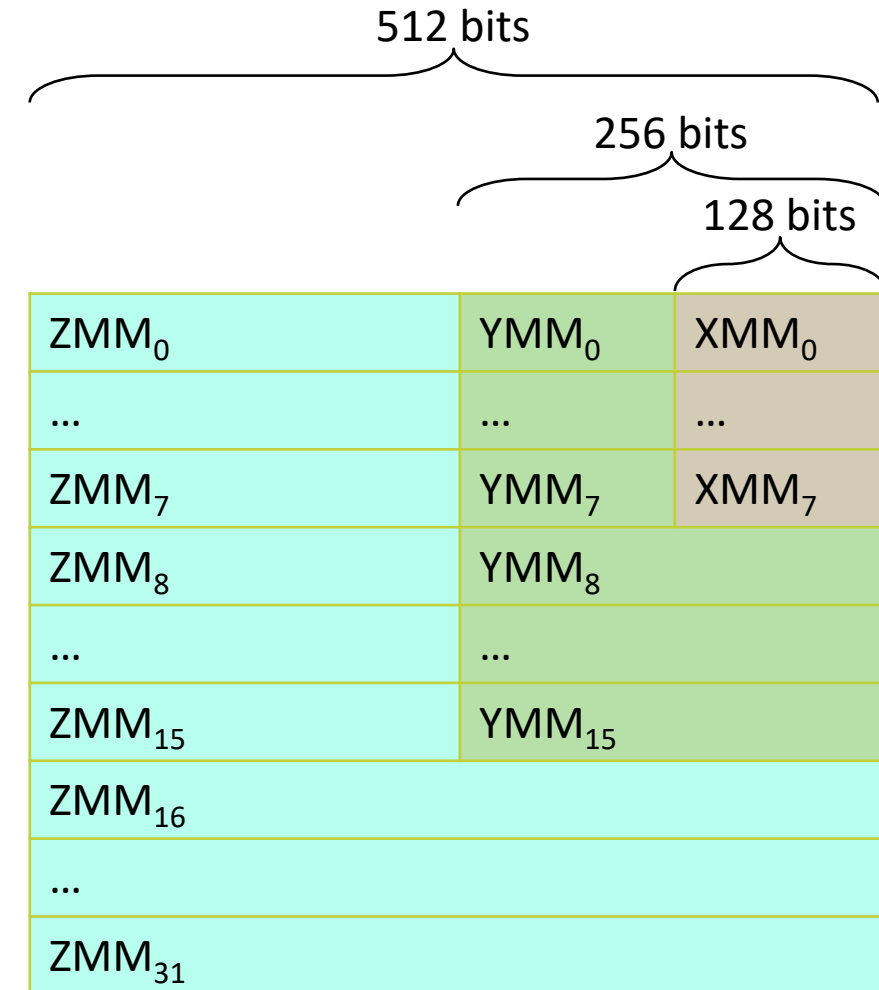
SIMD en la arquitectura x86

- **MMX (1996):** usa los registros FPU de 64 bits y contempla operandos de 8 o 16 bits
- **SSE (1999):** conjunto de 8 registros independientes de 128 bits que pueden actuar sobre operandos de 8, 16 y 32 bits
- **SSE2/3/4 (2001-2007):** se agregan operaciones de punto flotante, de forma que es posible operar sobre cuatro elementos de simple precisión o dos de doble precisión
- **AVX/AVX2 (2010-2013):** amplía a 16 registros de 256 bits, aunque reduce los tipos de operandos a ocho enteros de 32 bits o cuatro flotantes de 64 bits
- **AVX-512:** duplica de nuevo el número de registros y su tamaño respecto a AVX

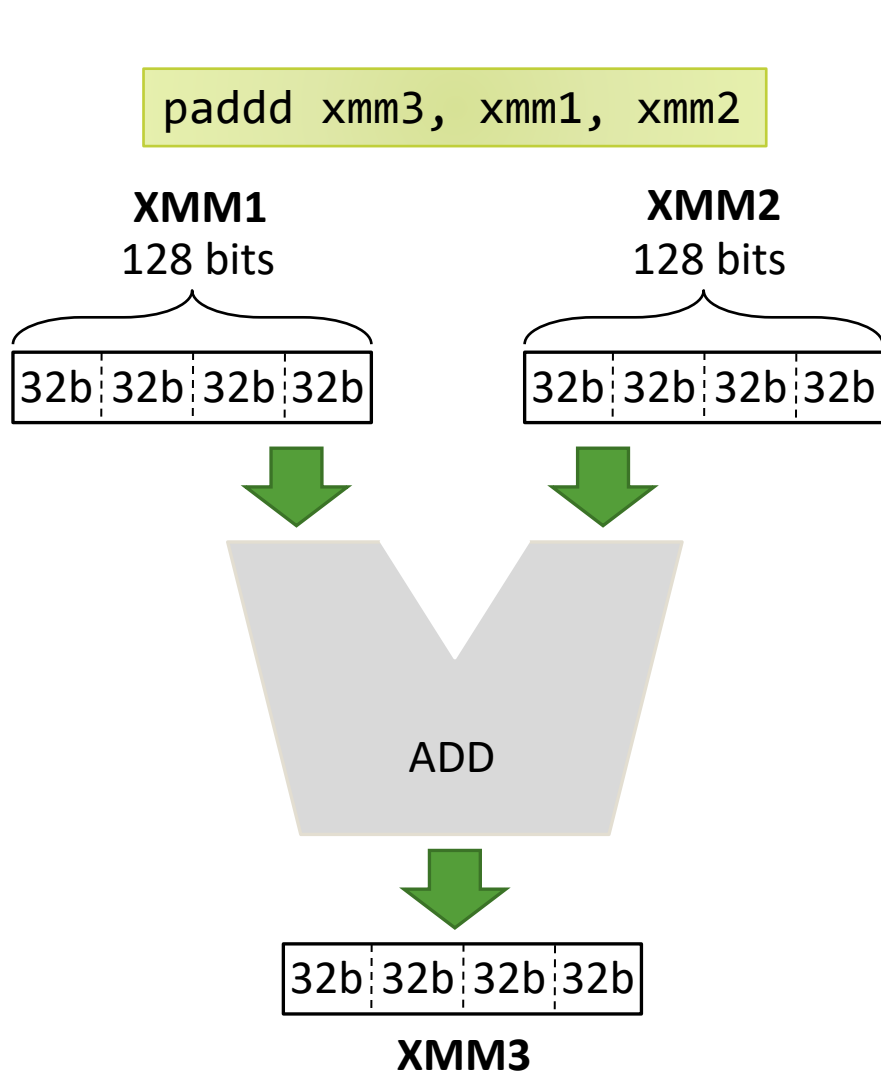
Registros SIMD

Características

- **Número:** el banco de registros SIMD tiene un tamaño variable, según la versión de las extensiones MMX/SSE/AVX con que cuente el procesador
 - 8 registros en SSE, 16 en AVX, 32 en AVX-512
 - La denominación también cambia: XMM_n en SSE, YMM_n en AVX, ZMM_n en AVX-512
- **Tamaño:** también difiere según la versión de las extensiones
 - 128 bits en SSE, 256 bits en AVX y 512 bits en AVX-512
 - No todos los procesadores modernos soportan todas las extensiones
- **Estructura:** los tipos de datos que pueden contener los registros varían según la versión de las extensiones
 - Habitualmente se soportan enteros y coma flotante de entre 8 y 64 bits



Unidades funcionales



PADDD (*Packet ADD Double-Word*)

XMM1	Sa	Sb	Sc	Sd
	+	+	+	+
XMM2	Sp	Sq	Sr	Ss
XMM3	Sa+Sp	Sb+Sq	Sc+Sr	Sd+Ss

- La extensión SSE incorpora registros **XMM** de 128 bits, así como una ALU capaz de operar sobre operandos de ese tamaño
- En un registro XMM se introducen cuatro enteros de 32 bits con el objetivo de operar sobre ellos de forma simultánea
- La ALU de SSE realiza una suma con dos operandos de 128 bits, pero desactivando el acarreo en el límite de cada bloque de 32 bits

Vectorial vs SIMD

Principales diferencias

- En SIMD el tamaño de los vectores sobre los que se opera viene impuesto por el tamaño de los registros y las divisiones que pueden hacerse. Un procesador **vectorial es más flexible** y puede procesar vectores de tamaño y estructura arbitrarios
- Las cargas y almacenamientos de SIMD han de realizarse **por completo antes de poder operar**, mientras que un procesador vectorial, merced a su segmentación profunda, puede cargarlo paso a paso en cada ciclo de reloj
- Con SIMD los vectores han de estar necesariamente **almacenados en posiciones de memoria consecutivas**. Los vectoriales contemplan mecanismos más sofisticados para operar con vectores dispersos o que tienen otra estructura
- El tipo y tamaño de los operandos en un registro SIMD están **codificados en la propia instrucción**, no hay registros de control vectorial, lo cual tiene consecuencias:
 - Se precisan muchas instrucciones distintas, según el tipo y tamaño de operando, lo cual hace más compleja la implementación
 - La última versión de las extensiones AVX ([Intel AVX10](#), julio 2023) cuenta con cientos de instrucciones SIMD cuya especificación ocupa más de 1300 páginas

GPU

Introducción



Introducción

Procesadores con un alto grado de paralelismo para ejecutar operaciones simples a fin de acelerar el cauce de procesamiento de gráficos

Conceptos

- Su origen está en los **aceleradores de gráficos** 3D que debían operar en paralelo sobre gran cantidad de vértices y píxeles, por lo que las GPU no comparten muchas de las características de las CPU
- Cuentan con decenas de CU (**Computing Units**), cada una con un gran número de núcleos (*cores*) que operan en paralelo y de forma sincronizada
- Cada CU capta una instrucción y la emite a todos sus núcleos, que la ejecutan cada uno de ellos sobre un dato distinto según un modelo SIMT (*Single Instruction Multiple Thread*)

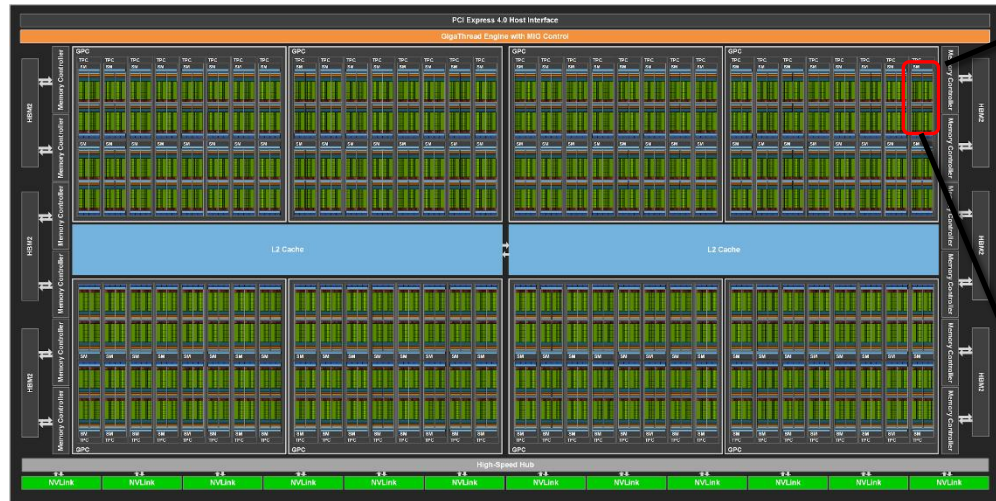
Arquitectura GPU (NVidia)

Terminología

- Cada CU se denomina **SM** (*Symmetric Multiprocessor*) y consta de distintos elementos
 - Un gran banco de **registros** y memoria caché L1
 - Su propio **contador de programa** y una unidad de emisión de instrucciones
 - Unidades de **carga y almacenamiento** de datos desde memoria
 - **Núcleos de cómputo** (*CUDA cores* en la denominación de NVidia) que contienen las ALU
- El **número de hilos** gestionados por un SM es habitualmente mucho mayor que el de núcleos disponibles
 - En las últimas generaciones (RTX 4090) cada SM cuenta con **128 núcleos** de procesamiento
 - Sin embargo es posible tener hasta **2048 threads** por SM
 - Para una GPU con 142 SM (RTX 4090Ti) habría un total de 18 176 núcleos y 290 816 hilos de ejecución
- El multiprocesamiento masivo compensa la **latencia del acceso a memoria**
 - Contar con un gran número de hilos activos y un banco de registros enorme facilita el cambio de contexto (*context switching*) sin pérdida de rendimiento
 - Los hilos se agrupan en *warps* que se activan de forma simultánea según la disponibilidad de sus datos

Arquitectura GPU (NVidia)

NVidia GA100 - 128 SM



[Alta resolución](#)

2048 threads por SM agrupados en 64 warps de 32 threads = 262 144 threads en total



64 cores por SM

8192 cores total

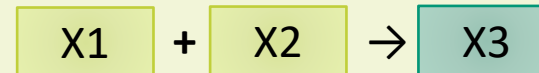
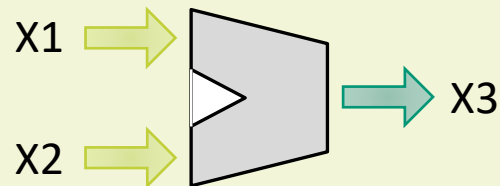
Escalar vs Vectorial vs SIMD vs GPU



Comparativa de arquitecturas

Escalar

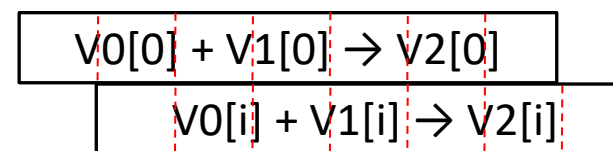
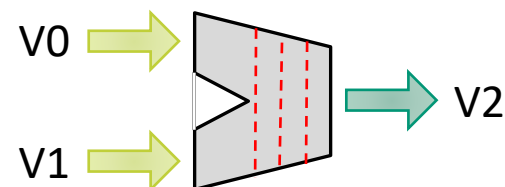
```
add x3, x1, x2
```



Suma de dos valores escalares

Vectorial

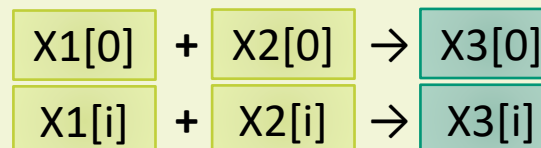
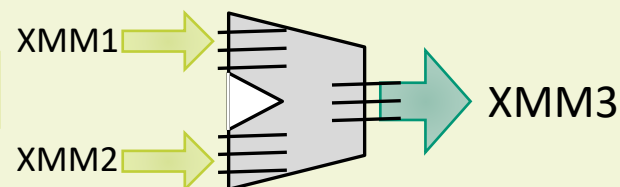
```
vadd.vv v2, v0, v1
```



Suma dos vectores de longitud variable con segmentación profunda

SIMD

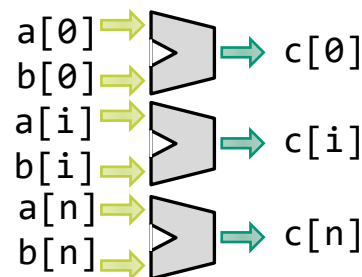
```
padd xmm3, xmm1, xmm2
```



Suma dos vectores de longitud fija de forma empaquetada

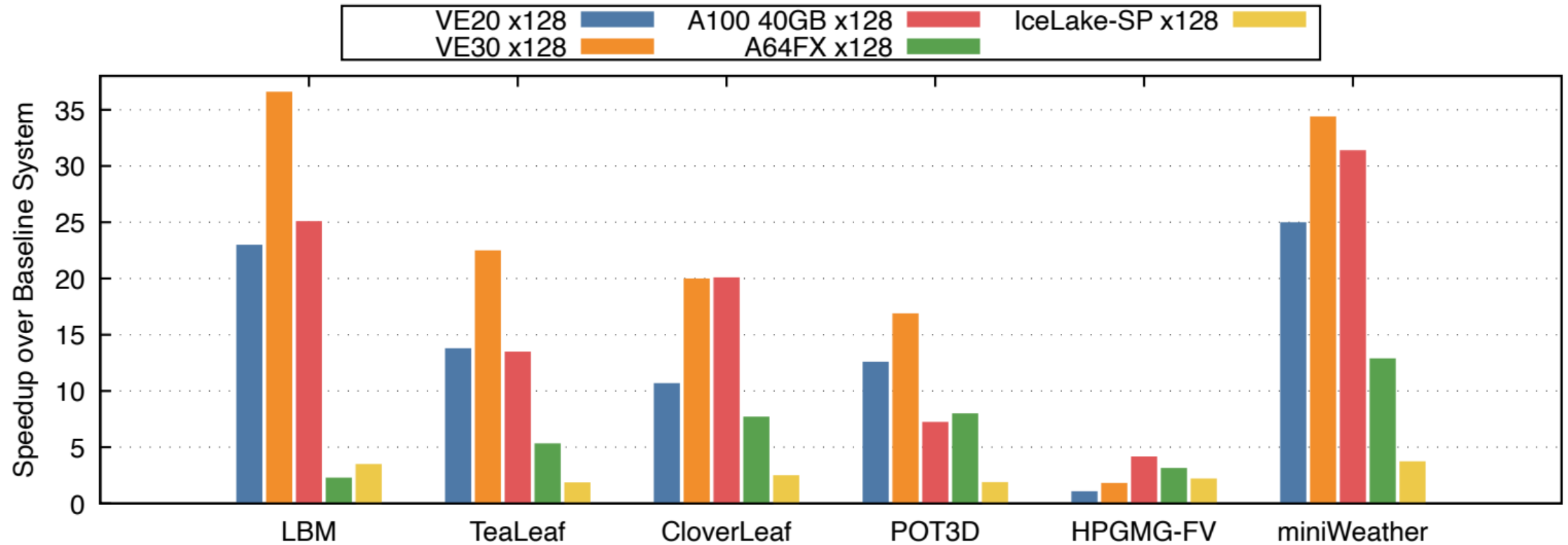
GPU

```
void add( int *a, int *b, int *c ) {  
    int tid = blockDim.x * blockIdx.x + threadIdx.x;  
    c[tid] = a[tid] + b[tid];  
}
```



Trata cada elemento de los vectores en un *thread* independiente, de forma que la suma de todos los elementos se ejecuta en paralelo (si hay núcleos suficientes para ello)

Comparativa de rendimiento



- VE20/VE30 - Procesador vectorial NEC SX-Aurora TSUBASA
- A100 40GB - GPU NVIDIA A100 (Ampere)
- A64FX - Procesador vectorial Fujitsu
- IceLake - Intel Xeon Platinum 8368 con extensiones AVX (SIMD)



TAKAHASHI, Keichi, et al. Performance Evaluation of a Next-Generation SX-Aurora TSUBASA Vector Supercomputer. En *International Conference on High Performance Computing*. Cham: Springer Nature Switzerland, 2023. p. 359-378.

Otras arquitecturas de paralelismo de datos



Arquitecturas para el paralelismo de datos

Aceleradores hardware

- Arquitecturas que aceleran algoritmos de IA (procesamiento de matrices)
 - Matrix Engine de IBM
 - IBM Power 10: 4 MMA (*Matrix Math Accelerator*), hasta 2 PB RAM, 18 000 millones de transistores, 2.48 TFLOPs
 - TPU (*Tensor Processor Unit*) de Google
 - Google TPU v5: 4 MXU (*Matrix-Multiply Units*), 16 GB HBM3, ?? transistores, 393 TFLOPs
 - Tensor core de Nvidia
 - NVidia B200 (*Blackell*): 16 896 núcleos, 192 GB HBM3, 208 000 millones de transistores, 10 PFLOPs
 - Matrix core de AMD
 - Instinct MI300: 19 456 núcleos, 192 GB HBM3, 153 000 millones de transistores, 3 967 TFLOPs
 - NPU (*Neural Processing Unit*) de Huawei
 - Huawei Ascend 310: 4 AI CPU, 18 GB memoria, ?? millones de transistores, 8 TFLOPs
 - WSE (*Wafer Scale Engine*) de Cerebras
 - Cerebras WSE-3: 900 000 núcleos IA, 44 GB SRAM (hasta 1.2 PB DRAM), 4 billones de transistores, 125 PFLOPs

Nota: el superordenador más rápido del mundo en el año 2000, el ASCI Red, alcanzaba un rendimiento de 1 TFLOP usando una arquitectura distribuida con múltiples computadores



2012

Cray XC30

100 Pflops (X 100 000 Cray 1)

<https://www.top500.org/lists/top500/2025/06>

2025

43 808 AMD EPYC 24C = 1 051 392 núcleos CPU

43 808 AMD MI300A = 9 988 224 núcleos GPU

1.742 Exaflops

29 581 KW

600 000 000\$



Otras arquitecturas avanzadas

Computación en memoria (*In-Memory computing*)

- El actual **cuello de botella** en el procesamiento masivo de datos es la **transferencia** de estos desde la memoria RAM (externa al propio procesador) a las unidades funcionales (internas al procesador) y de vuelta a la memoria
- Al integrar las unidades funcionales en los propios circuitos de memoria se evita la transferencia, y se **opera sobre los datos almacenados** en memorias diseñadas para este fin: MRAM, PCM, ReRAM, etc.

Computación analógica (*Analog computing*)

- Procesar grandes matrices de datos con circuitos digitales implica un **alto consumo de recursos**, especialmente energía
- Los diseños de nuevos procesadores analógicos permiten realizar esas operaciones, con una precisión aproximada, empleando mucha **menos energía y tiempo**. Lo habitual es combinar elementos digitales y analógicos
- Es una tecnología propuesta para el desarrollo de procesadores neuromórficos y que también se aplica en el desarrollo de memorias con capacidad de computación en memoria

Circuitos ópticos (*Photonic computing*)

- Los procesadores actuales operan con transistores electrónicos: se emplean **corrientes de electrones** para representar los estados digitales. Estos circuitos consumen mucha energía y disipan una cantidad considerable de calor
- Los circuitos ópticos basan su funcionamiento en el uso de **fotones** en lugar de electrones, por lo que precisan **menos energía** y producen mucho **menos calor**. Empleados ampliamente en equipos de comunicaciones (fibra óptica), se están aplicando también en la fabricación de procesadores con importantes ventajas

Cuestiones clave de este tema



Cuestiones clave

Qué deberías saber

*Al inicio de este tema se planteaban unos objetivos específicos que deberían permitirte **responder a las siguientes cuestiones clave***

Cuestiones

- ¿En qué consiste el paralelismo de datos?
- ¿Cuál es la diferencia entre paralelismo de datos y de instrucciones?
- ¿Cómo trabaja un procesador vectorial con segmentación profunda?
- ¿En qué se basan las extensiones *packed* SIMD de los procesadores modernos?
- ¿Cuáles son las ideas básicas sobre el funcionamiento de las GPU?
- ¿Cómo se tratan los datos en cada una de las arquitecturas descritas?

Material adicional

Descripción

Para ampliar tus conocimientos sobre los contenidos de esta semana te recomendamos que consultes los recursos indicados a continuación.

Recursos

- **Capítulo 18** (*Procesamiento paralelo*) del libro **Organización y arquitectura de computadores 7ED** disponible en [formato digital](#) en la BUJA (recuerda identificarte para poder acceder a leerlo desde tu navegador)
- **Capítulo 4** (*Data-Level Parallelism in Vector, SIMD, and GPU Architectures*) del libro **Computer Architecture 5ED** disponible en [formato digital](#) en la BUJA (recuerda identificarte para poder acceder a leerlo desde tu navegador)
- **Capítulo 6** (*Parallel Processors from Client to Cloud*) del libro **Computer Organization and Design** disponible en la Biblioteca de la UJA ([enlace](#))